Revision chapter 1

- Answer the following question among them

1. What are translators?
2. What is compiler? Explain about the phases of compiler?
3. Explain about symbol table and error handling?
4. Define pre-processor, assembler, linker and loader?
5. What are the similarity and difference between Inter prater and Compiler?
6. Explain about single-pass and multi-pass
7. Explain about the architecture of compiler design?
8. Explain about compilation and execution?
9. Write about language processing system with diagram?

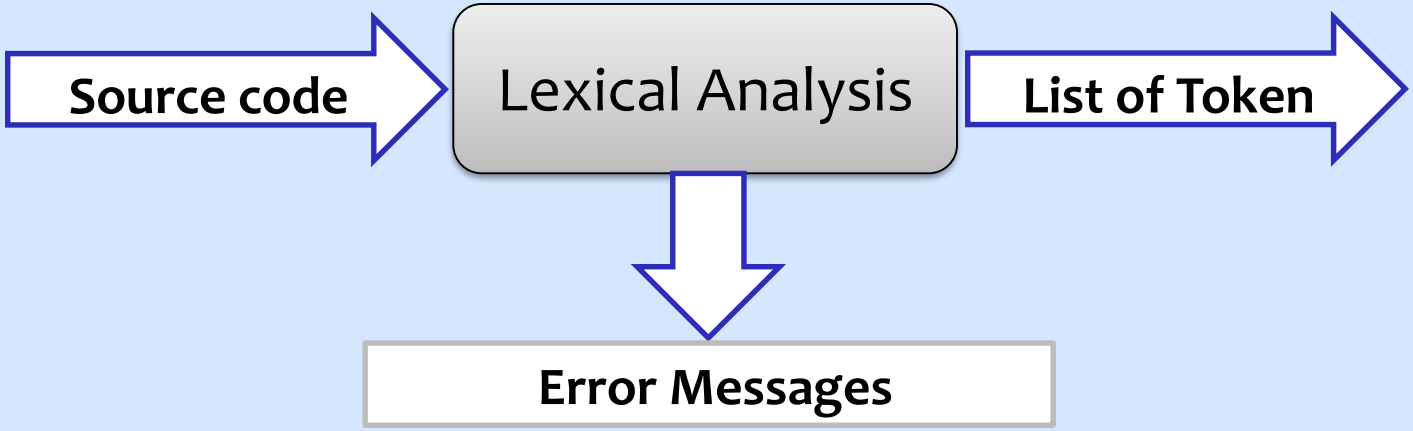
Chapter 2 Outline

- **Lexical Analysis?**
  - Role of lexical analysis
  - Issue in lexical analysis
  - Token ,Patter, Lexeme
  - Attributes of a token
- **Input Buffering**
  - Buffer pairs
  - Sentinels (Guards)
- **Specification of token using Regular expression**
  - Regular Expression
- **Recognizing of token using Transition diagram**
  - Transition diagram

Lexical Analysis

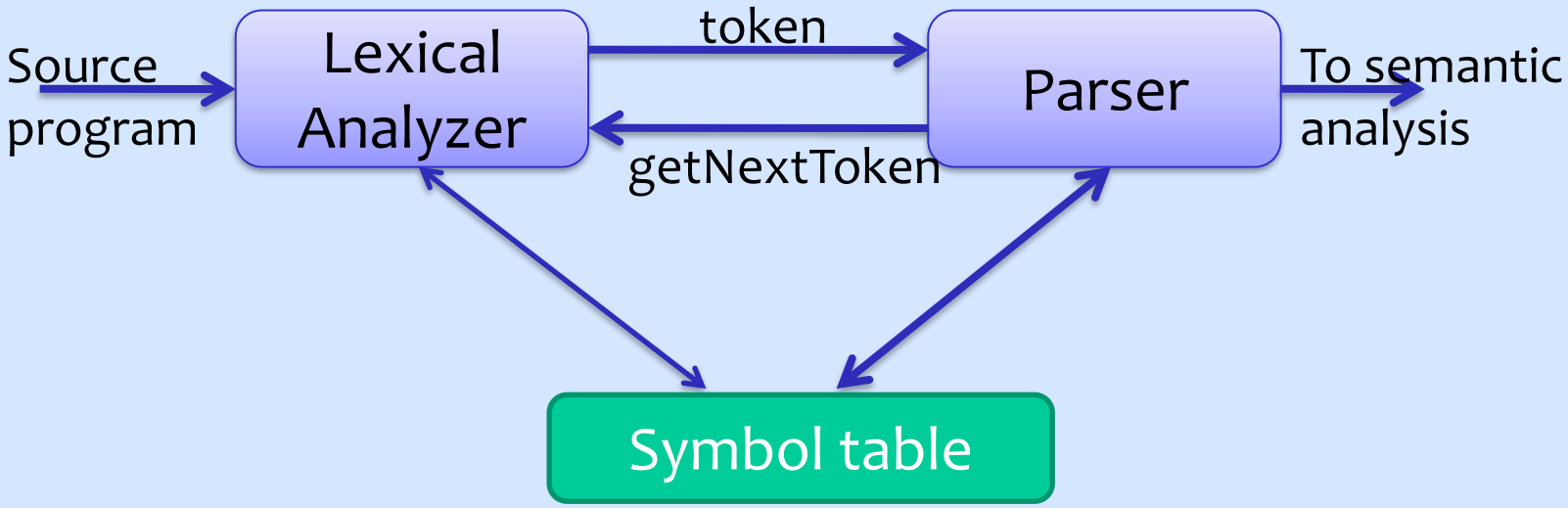
Lexical (Linear) analysis or scanning

- The first phase of a compiler, This phase is the initial part of reading and analyzing the program text: The text is read and divided into **tokens**, is a smallest unit of a program.
- so the source code translated in to a list (stream) of token.by removing white space and comments



Role of Lexical Analysis

- **Main task**
  1. To read the input characters
  2. To group them into lexemes
  3. To produce a sequence of tokens used by the parser for syntax analysis As an assistant of parser
  4. To Interact with the symbol table
     - To Insert identifiers



Role of Lexical Analysis

Simple task

5. To strip out (remove)
   - Comments : such as // or /* -------*/
   - Whitespaces : such as blank, newline, tab, …

   - To correlate error messages generated by the compiler with the source program
   - To keep track of the number of newlines seen
   - To associate a line number with each error message

Sample of **A lexical error** is

- ✓ A mistake in a lexeme,
  - for examples, typing far instead of for,
- ✓ Unclosed comment
  - For example /* read a value….

Role of Lexical Analysis

- Sometimes lexical analyzers are divided in two a cascade of two phases

1. Lexical analyzer: - to produce a sequence of tokens as an output.(main task)
2. Scanning :-the scanner is responsible for simple task

Issue in lexical analysis

- There are several reasons for separating the analysis phase into lexical and syntax analysis.

1. Simplicity of design: is perhaps the most important consideration
2. Improved compiler efficiency: liberty to apply specialized techniques that serves only lexical tasks,
3. Enhanced compiler portability:- Input device peculiarities are restricted to the lexical analyzer

Token, Lexeme, Pattern:

Token: Token is a sequence of characters that can be treated as a single logical entity or a smallest unit of a program.

- There are two categorized of token those are:-
  1. Valid Tokens
  2. Invalid tokens

Typical valid tokens are	Typical invalid tokens are
Keyword	White Space (blank)
Identifiers	New line (/n)
Operators	Tab space (/t)
Special symbols	Comment // or /* -------*/
Number	Pre-processor directives # include<header.h>
Literal/Constant	Pre-processor directives # define avg(a+b/2)

Token, Lexeme, Pattern:

Pattern: A set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token. Patter uses a regular expression to identify tokens.

Lexeme: A lexeme is a sequence of characters in the source program that is by the pattern for a token.

Example: Description of token

Token	Lexeme	Pattern
Keyword(if)	If	characters i, f
Id	X	Letter followed by seq. of alphanumeric l (l+d)*
Relation Operator	<,<=,=,<>,>=,>	< or <= or = or < > or >= or lette followed by letters & digi (l+d)*
Special symbols	:,:,(,),{,},',",.,	: or : or (or) or { or } or' or " or . or ,
Number	3.14	Any numeric constant
Literal/Constant	"core"	Anything but ", surrounded by "

Attributes of a token

- When two or more lexemes match the pattern for a token then the lexical analyzer must provide additional information with the token.
- Additional information is placed in a symbol-table entry and the lexical analyzer represents these lexemes in the form of tokens as:**<token-name, attribute-value>**
- ✓ Token-name**:** an abstract symbol is used during syntax analysis,
- ✓ Attribute-value**:** points to an entry in the symbol table for this token.

E.g. E = M * C ** 2 has 7 tokens and associated attribute-values:

E :- <(id,1) reference to symbol-table entry for E>

= :- <assign_op,>

M :- < (id,2) reference to symbol-table entry for M>

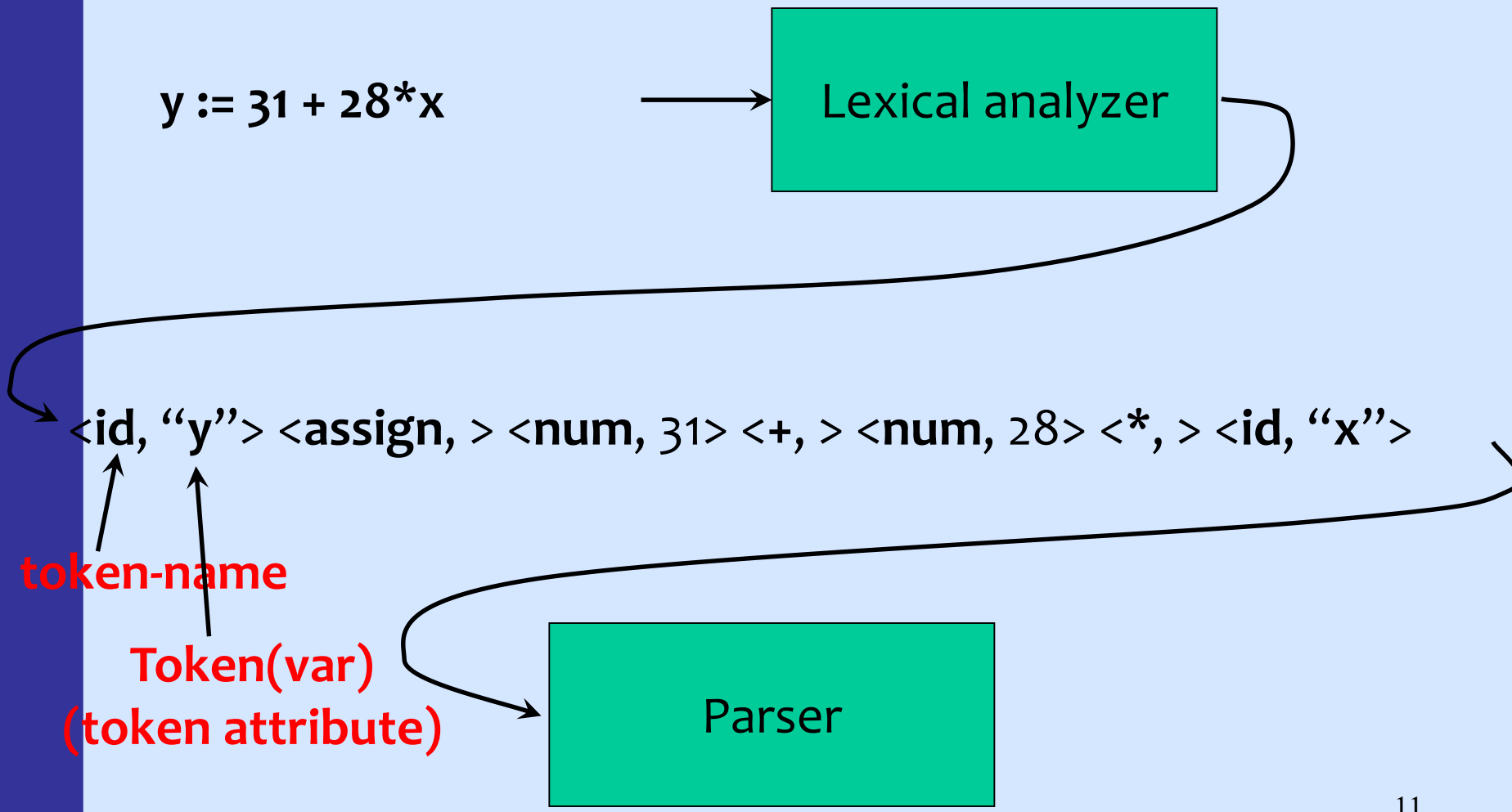
***** :- <mult_op,>

C :- < (id,3) reference to symbol-table entry for C>

****** :- <exp_op,>

2 :- <num, integer value 2>

Attributes of a token



Some example of token

C++ Program :- Hello world

1. #include <iostream.h>
2. #include <conio.h>

Handle by Preprocessor

3. void main()

1	2	3	4
void	main	(	)

4. // his is a hello world program → **X Comment, Invalid token**

5. {

5
{

6. cout << “Hello World!\n";

6	7	8	9	10
cout	<	<	“Hello world!\n”	;

7. }

11
}

❖ So The total No of token is 11

Token of a Hello World program

void	main	(	)
{	cout	<	<
“Hello world!\n”	;	}	

Keyword	Identifier	Numbers	Literal/ Constant	Special symbols	Operators
void	main	-	“Hello world!\n”	(	<<
cout	-	-	-	)	-
-	-	-	-	{	-
-	-	-	-	}	-
-	-	-	-	;	-
2	1	0	1	5	1

Some example of token

C++ Program :- Adding two numbers &multiply by 10^2

```
1. #include <iostream.h>
2. #include <conio.h>
3. void main()
4. {
5. int a, b, c;
6. cout << "Enter two numbers to add\n";
7. cin >> a >> b;
8. c = a * b+10^2;
9. cout <<“The value of c = " << c << endl;
10. }
```

Handle by Preprocessor

void	main	(	)			
{						
int	a	,	b	,	c	;
cout	<	<	"Enter two .."			;
Cin	>	>	a	>	>	b	;
c	=	a	*	b	+	10	^	2	;
cout	<	<	“The…"			
<	<	c	<	<	;	
}						

- So The total No of token is 46

Tokens of Adding two Numbers

Keyword	Identifier	Numbers	Literal/ Constant	Special symbols	Operators
void	a	10	“Enter...”	(	<<
int	b		“Sub....”	)	>>
cout	c			{	+
cin	main			}	*
				;	=
				,	
4	4	1	2	6	5

operators

A.O	R.O	L.O	Unary. O	Assi. O	B.O
+				=	<<
*					>>
					^
2	0	0	0	1	3

Some example of token

C++ Program :- G.C.F of two Numbers

❖ So The total No of token is **60**

Handle by Preprocessor

```
1.  #include <iostream.h>
2.  #include <conio.h>
3.  void main()
4.  {
5.  int  n,m, r;
6.  cout << "Enter two No to get G,C,F\n";
7.   cin >> n >> m;
8.   while (n!=0)
9.  {
10. r = n % m;
11. m=n;
12. n=r;
13. }
14. cout <<“G.C.F of entered numbers = " << m << endl;
15. }
```

void	main	(	)

{

int	m	,	n	,	r	;

cout	<	<	"Enter two ….."	;

Cin	>	>	m	>	>	n	;

while	(	n	!	=	0	)	;

r	=	n	%	m	;

m	=	n	;

n	=	r	;

{

}

cout	<	<	“Sum."

<	<	M	<	<	;

}

E.g. Token of a G.C.F program

Keyword	Identifier	Numbers	Literal/ Constant	Special Symbols	Operators
void	n	o	“Enter...”	(	<<
int	m		“C.F.G....”	)	>>
cout	r			{	%
cin	Main			}	!=
while				;	=
				,	
4	4	1	2	6	5

operators

A.O	R.O	L.O	Unary. O	Assi. O	B.O
%	!=			=	<<
					>>
2	1	0	0	1	2

Input Buffer

- The Input buffer is also commonly known as the **input area or input block.** When referring to computer memory,
- **An input buffer** is a location that holds all incoming information before it continues to the **CPU for processing**.
  - This is the **only part** of the **compiler** that **examines** the entire **input program** one **character at** a **time**
  - The scanner accounts for ~**25-30%** of total compile time.
- There are three general approaches to the implementation of a lexical analyzer:

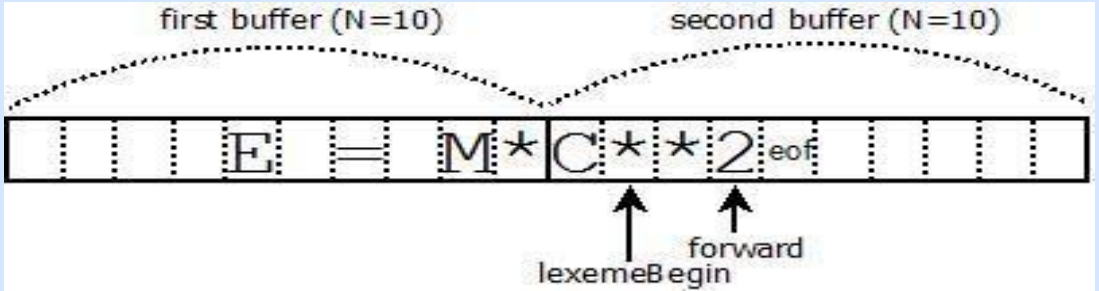
1. **Use a lexical-analyzer generator,** such as Lex or Flex compiler to produce the lexical analyzer from a regular expression
2. **Write the lexical analyzer in a conventional systems-programming language,** using I/O facilities of that language to read the input.
3. **Write the lexical analyzer in assembly language** and explicitly manage the reading of input.

Input Buffer

- To speed up reading the source program, input buffer using d/t technique
  - one buffer technique, the last lexeme under process will be over-written when we reload the buffer.
  - Two-buffer scheme (Buffer pair) handling large look ahead safely
  - sentinels –improvement which saves time checking buffer end
- There are two pointers to the input are maintained
  1. lexemeBegin – marks the beginning of the current lexeme
  2. forward – scans ahead until a pattern match is found
  - E.g. + , - , = , < , > , …….
  - i++ , i-- , a == b, a += b, a *= b, a /= b, a -= b, a <= b, a >= b

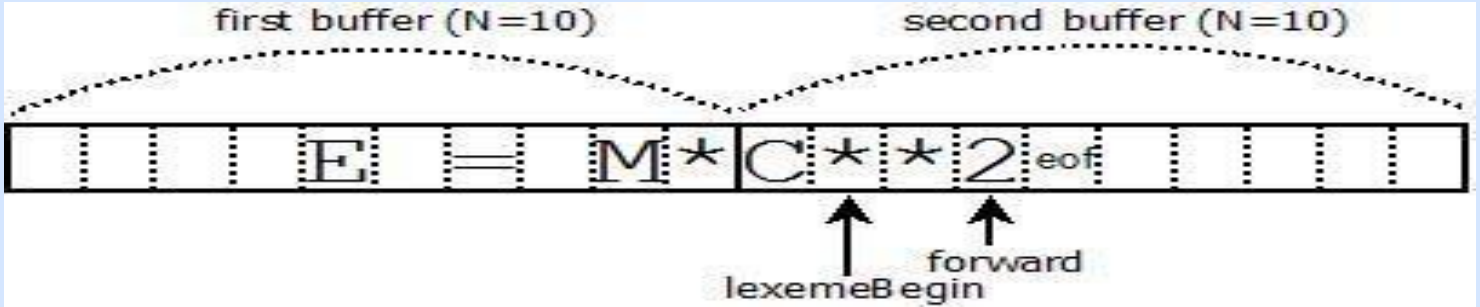
Buffer Pair

- Buffer pairs:- The input buffer has two halves with N characters in each half.
- N might be the size of a disk block like 1024 or 4096.
- Mark the end of the input stream with a special character eof.
- Maintain two pointers into the buffer marking the beginning and end of the current lexeme.
- two pointers to the input are maintained
  1. lexemeBegin – marks the beginning of the current lexeme
  2. forward – scans ahead until a pattern match is found



Buffer Pair

- Buffer pairs:- The input buffer has two halves with N characters in each half.
- N might be the size of a disk block like 1024 or 4096.
- Mark the end of the input stream with a special character eof. Maintain two pointers into the buffer marking the beginning and end of the current lexeme.
- two pointers to the input are maintained
  1. lexemeBegin – marks the beginning of the current lexeme
  2. forward – scans ahead until a pattern match is found



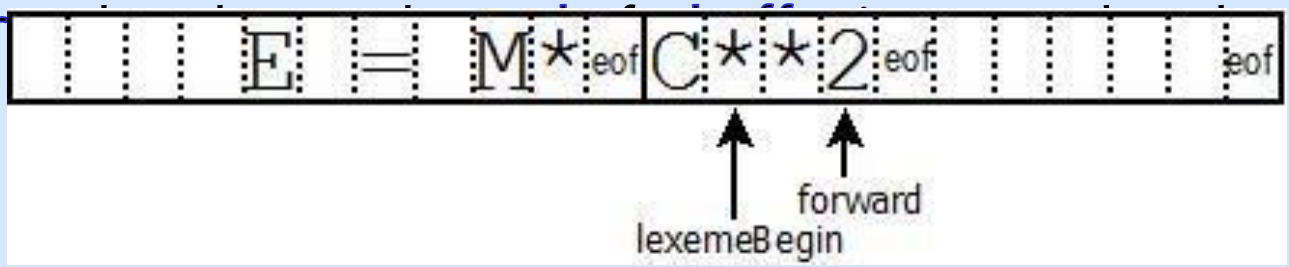
Simple Input Buffering algorithm

- Initially, both pointers point to the first character of the next lexeme to be found.
- The forward pointer is scanned ahead until a match for a pattern is found. After the lexeme is processed set both pointers to the character following the lexeme.
- Code to advance forward pointer:

```
if forward at end of first half then
        reload second half;
        forward := forward + 1
else if forward at end of second half then
        reload first half;
        move forward to beginning of first half
else forward := forward + 1;
end if
```

Sentinels (Guards)

- ❖ Forward pointer
  - to test if it is at the end of the buffer
  - to determine what character is read (multi way branch)
- ❖ Sentinel:- added eof at each buffer end
- Objective: Optimize the common case by reducing the number of tests to one per advance of fwd.
- Idea: Extend each buffer half to hold a sentinel at the end.
- This is a special character that cannot occur in a program (eof)
- It signals the need for some special action (fill other buffer-half, or terminate processing).
- when appear
  input is at an



Simple Sentinels algorithm

- For almost every input character perform three tests:
  1. Is the character an eof?
  2. Is the pointer at the end of the first half ?
  3. Is the pointer at the end of the second half?
- This can be reduced to one test per character by using sentinels.
- Add an eof character past the end of each buffer half.
- Use the code from next slide to advance the forward pointer.

```
forward := forward + 1;
if forward = eof then
   if forward at end of first half then
      reload second half;
       forward := forward + 1   end
else if forward at the second half then
   reload first half;
   move forward to beginning of first half
   end
else terminate lexical analysis   end;
```

Specification of Tokens

- There are Two Dual Notions to specification and recognition of tokens.
  1. Generative approach
     - (grammar or regular expression)
  2. Recognition approach
     - (automaton)

- Many theorems to transforms one approach automatically to another

Specification of Tokens

- Regular Definition of Tokens: - defined in regular expression
- Regular expression is an important notation for specifying patterns.
- Each pattern matches a set of strings, so regular expressions serve as names for a set of strings.
- Programming language tokens can be described by regular languages.

  E.g. **letter** →A|B|…|Z|a|b|…|z

  digit →0|1|2|…|9

  Id → **letter**(**letter**|**digit**)*

  Id →{x ,y, y, count, d12, h45r5}
- The specification of regular expressions is an example of a recursive definition.
- Regular languages are easy to understand and have efficient implementation.

Specification of Tokens

- Let us understand how the language theory undertakes the following terms, ***Terminology:***

Symbol

- An abstract entity that we shall not define formally

e.g. (such as "point" in geometry) Letters, digits and punctuation is examples of symbols.

Alphabets

- A finite set of symbols out of which we build larger structures.
- An alphabet is typically denoted using the Greek word sigma ∑,

e.g., ∑ = {0, 1} is a set of binary alphabets,

∑ = {0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F} is a set of Hexadecimal alphabets,

∑ = {a-z, A-Z} is a set of English language alphabets.

Strings

- Any finite sequence of alphabets is called a string.
- Length of the string is the total number of occurrence of alphabets,

Specification of Tokens

- Empty (zero length) string denoted by a Greek word epsilon ε ,
- Length of string denoted in absolute value | |:
- e.g. 0010, |0010| = 4, 007,|007| = 3, ε,|ε|=0

Language

- is considered as a finite set of strings over some finite set of alphabets.
- Computer languages are considered as finite sets, and mathematically set operations can be performed on them.
- Finite languages can be described by means of regular expressions.
    - Empty set is a language
    - {ε} The set containing empty string is a language
    - The set of all possible identifier is a language

      e.g. r = {ab} => L(r) => L(ab)

Specification of Tokens

Operations on Languages

- There are several important operations that can be applied to languages.
- Let L be the set of {a,b,c} and M be the set of {1,2}

1. Union: - L U M { x | x is in L or in M} E.g. L UM = {a,b,c,1,2}
2. Concatenation :- LM :- {st| s is in L and t is in M}
   E.g. L M =$\{a1,a2,b1,b2,c1,c2\}$
3. Kleen closure :- $L^* = \cup_{i=0,\ldots,\infty} L^i$, means zero or more concatenation of L.
   E.g. L* is the set of all string of letter and empty set,
   $\{\varepsilon\}$ or {a} or {a,b} or {a,b,c},
4. positive closure :- $L^+ = \cup_{i=1,\ldots,\infty} L^i$ means one or more concatenation of L.

E.g. L^+ is the set of all string of one or more latter without empty set {a} or {a,b} or {a,b,c}

5. Exponential :- L^n , n is any number of integer

E.g. $L^0 = \{\varepsilon\}$; ,L^2 ={2-latter string} = {aa,ab,ac,ba,bb,bc,ca,cb,cc… }

Regular Expression

Regular Expressions

- are a Set of Rules / Techniques for Constructing Sequences of Symbols (Strings) From an Alphabet.
- Regular expressions are an algebraic way to describe languages.
- The grammar defined by regular expressions is known as regular grammar.
- The language defined by regular grammar is known as regular language.

Regular Expression & Regular language

- Regular Expression: - A notation (rule) that allows us to define a pattern in a high level language.
- Regular language:- Each regular expression r denotes a language L(r) (the set of sentences relating to the regular expression r)

Notes: Each word in a program can be expressed in a regular expression and regular expression denotes a language.

Regular Expression

The rule of regular expression over alphabet Σ

The rule of regular expression can be categorized in to Basic symbol and Basic operation

1. Basis symbols rule:
   - ε is a regular expression denoting language $\{\varepsilon\}$
   - $a \in \Sigma$ is a regular expression denoting $\{a\}$
2. Basic operation

Suppose r and **s** are regular expressions denoting languages L(r) and M(s) respectively then

- $r \mid s$ is a regular expression denoting $L(r) \cup M(s)$
- r s is a regular expression denoting L(r)M(s)
- **r*** is a regular expression denoting L(r)*
- r^+ is a regular expression denoting $L(r)^+$

Note:- A language defined by a regular expression is called a regular set

Regular Expression

Precedence and Associativity

- The precedence of regular expression are Closer (*), concatenation (.), and pipe sign (|) are left associative
  - Closer* has the highest precedence
  - Concatenation (.) has the second highest precedence.
  - Pipe sign (|) has the lowest precedence of all.

Note: - All are Left-Associative. Parentheses are dropped as allowed by precedence rules

Example Let r is a set of {1,2}
Let s is a set of {a, b}
Let t is a set of {7,f}
(s | t r *) first r* second tr* atlast s|tr*

Algebraic Properties of RE

AXIOM	DESCRIPTION
$r \mid s = s \mid r$	$\mid$ is commutative
$r \mid (s \mid t) = (r \mid s) \mid t$	$\mid$ is associative
$(r\ s)\ t = r\ (s\ t)$	concatenation is associative
$r\ (s \mid t) = r\ s \mid r\ t$ $(s \mid t)\ r = s\ r \mid t\ r$	concatenation distributes over $\mid$
$\in r = r$ $r \in = r$	$\in$ Is the identity element for concatenation
$r^* = (r \mid \in)^*$	relation between * and $\in$
$r^{**} = r^*$	* is idempotent

Note:- A language may be represented by two or more equivalent regular expressions. idempotent operation is one that has no additional effect if it is called more than once with the same input parameters.

Notational shorthand's

- Certain constructs occur so frequently in regular expressions that it is convenient to introduce notational shorthand for them.
- **One or more instances :-** The unary postfix operator + Means "one or more instances of " if r is a regular expression that denotes the language L(r), then (r)+ is a regular expression that denotes the language (L(r))+ means.

  $(r)^+$ denoting $(L(r))^+$

  $(r)^+$ digit$^+$
- **Zero or More instance :-** the * operator

R* is shorthand for $\varepsilon|r^+$

- **Zero or one instance**

r? is a shorthand for $r|\varepsilon$ (E(+|-)?digits)?

- **Character classes**

[a-z] denotes a|b|c|…|z [A-Za-z] [A-Za-z0-9]

Notational shorthand's

Representing valid tokens of a language in regular expression with example

If x is a regular expression, then:

- x* means zero or more occurrence of x.

i.e., it can generate { e, x, xx, xxx, xxxx, … }

- x+ means one or more occurrence of x.

i.e., it can generate { x, xx, xxx, xxxx … } or x.x*

- x? means at most one occurrence of x

i.e., it can generate either {x} or {e}.

[a-z] is all lower-case alphabets of English language.
[A-Z] is all upper-case alphabets of English language.
[0-9] is all natural digits used in mathematics.

.

Notational shorthand's

Representing occurrence of symbols using regular expressions

- letter = [a – z] or [A – Z]
- digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 or [0-9]
- sign = [+ | -]

Representing language tokens using regular expressions

- Digit = (digit)$^+$
- Letter = (letter)$^+$
- Identifier = (letter)(letter | digit)*
- Decimal = (sign)?(digit)+
- Real Number → ([digit] + "." [digit] *) | ("." [digit] +)
- keyword -> keyword
- Relation operator -> < | > | <= | >= | = | <>

The only problem left with the lexical analyzer is how to verify the validity of a regular expression used in specifying the patterns of keywords of a language. A well-accepted solution is to use finite automata for verification